RT-2-Inspired Language-Conditioned Pick-and-Place in PyBullet

Technical Project Report

1. Title, Abstract & Executive Summary

Abstract

This report documents a single-file robotics simulation that translates free-form natural-language commands into pick-and-place behavior on a simulated KUKA LBR iiwa arm. The system is built in PyBullet and is *inspired by* the Vision-Language-Action (VLA) paradigm of RT-2. Instead of a monolithic VLA that emits discretized action tokens, it uses a modular pipeline. A regular expression splits a command into a *pick* fragment and a *place* fragment. A Sentence-Transformer model (`all-MiniLM-L6-v2`) embeds each fragment and matches it by cosine similarity against textual descriptions of the visible objects. Classical inverse kinematics (IK) with position control and a fixed-constraint grasp then executes the motion.

Executive Summary

- **Objective:** Let a user type commands such as "pick up the red cube and place it on the blue cube" and have the simulated arm carry them out, without hardcoded command strings.
- **Method:** Camera and segmentation capture determine which objects are visible. Language grounding is done by semantic similarity. Motion is generated by IK and executed under PD position control.
- **Key takeaway:** The project is a useful, low-cost testbed for the *language-to-object grounding* part of a VLA stack, and its interfaces (command → target object → end-effector waypoints) are clean enough to swap in a real VLA later.
- **Important caveat:** This is not an RT-2 implementation. It contains no vision-language-action transformer, no action tokenization, and no learned policy. Visual input is captured and segmented, but object identity currently comes from simulator metadata (color names), not from pixels.

2. Introduction & Theoretical Background

2.1 Vision-Language-Action Models

Classical robot learning maps observations to actions with policies trained from scratch on robot data. This is data-hungry and generalizes poorly outside the training distribution. VLA models take a different route. They start from a large vision-language model (VLM) pretrained on web-scale image-text data and adapt it to output robot actions.

2.2 RT-2

RT-2 (Google DeepMind, 2023) co-fine-tunes a pretrained VLM (e.g., PaLI-X, PaLM-E) on both web-scale vision-language tasks and robot trajectory data. Its central ideas are:

1. **Actions as tokens.** Continuous end-effector deltas (position, rotation, gripper) are discretized into bins, e.g., 256 per dimension, and written as text tokens. The model generates them like any other language output.
2. **Co-training.** Joint training on web data and robot data keeps the semantic knowledge of the VLM while grounding it in control.
3. **Emergent generalization.** The model can follow instructions involving unseen objects, symbols, or reasoning (e.g., "pick the object that could be used as a hammer"), because the semantics come from web pretraining rather than robot demonstrations.

2.3 Where This Project Sits

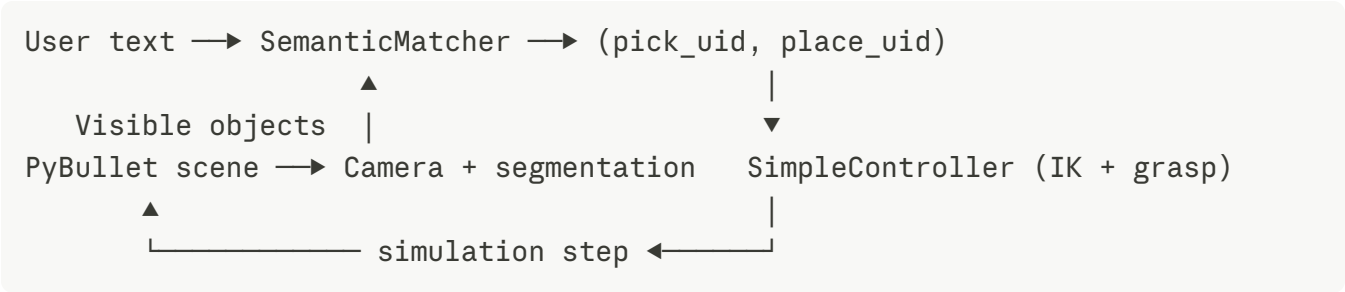
The present system borrows the *spirit* of RT-2 — leverage pretrained language representations so that commands need not match a fixed vocabulary — but implements it modularly:

Aspect	RT-2	This project
Perception	VLM visual encoder over RGB images	Segmentation mask used for visibility; names from simulator metadata
Language understanding	Joint VLM transformer	Regex phrase extraction + Sentence-Transformer similarity
Action generation	Autoregressive action tokens	Analytic waypoints + IK

Aspect	RT-2	This project
Learning	Co-fine-tuned on robot and web data	None for control; pretrained embeddings only
Generalization	Emergent, open-vocabulary	Limited to paraphrases of known object names

3. System Design & Architecture

The program is a single Python file composed of five logical layers.



3.1 Scene Construction (`create_scene`)

Loads a ground plane and a fixed-base KUKA iiwa (`kuka_iiwa/model.urdf`). Three 6 cm cubes (red, green, blue, 0.1 kg) are placed in a row at x = 0.55 m, y ∈ {−0.15, 0, 0.15}. A dictionary maps each PyBullet body ID to its color name and RGBA value.

3.2 Observation Processing (`get_camera_image_and_seg`)

A virtual camera is defined with `computeViewMatrixFromYawPitchRoll` (target [0.6, 0, 0.05], distance 0.9 m, yaw 90°, pitch −20°) and a 60° field-of-view projection. A 320×240 frame is rendered with `getCameraImage` , returning RGB pixels and a per-pixel segmentation mask in which each value is a body unique ID. The main loop uses the mask to determine which cubes are visible; an object that is fully occluded or out of frame is excluded from matching.

A helper, `seg_to_object_crops` , extracts padded tight bounding-box crops per object from the mask. It is implemented but not yet called, and is the natural hook for visual (e.g., CLIP) grounding.

3.3 Language Grounding (`SemanticMatcher`)

The matcher performs three steps.

1. **Phrase extraction.** Two regular expressions capture the noun phrase after a pick-type verb (*pick, grab, take, lift*) and after a place-type verb (*place, put, set*), optionally followed by a preposition (*on, onto, in, into, at*). Matching stops at *and*, *then*, or a comma.
2. **Candidate generation.** For each visible object, four textual descriptions are generated from its name, such as "red cube", "red block", "a red object", and "red colored cube". This is a lightweight form of prompt ensembling that improves robustness to synonyms.
3. **Similarity scoring.** The full command, the pick fragment, the place fragment, and all candidates are encoded in a single batch. Each fragment is assigned to the object whose best candidate has the highest cosine similarity.

If either fragment is missing, a fallback ranks objects by the maximum similarity between the full command and each object's candidates. The top object becomes the pick target and the second becomes the place target. The function returns `(pick_uid, place_uid, debug_scores)`, so the decision is inspectable.

3.4 Control (`SimpleController`)

The controller exposes three routines.

- `move_to(target_pos, target_ori, steps)`: At every simulation step, calls `calculateInverseKinematics` for the end-effector link (index 6) and commands each non-fixed joint with `POSITION_CONTROL` (force 200 N·m). The default orientation is a 180° pitch, pointing the tool downward.
- `pick(obj_id)`: Moves to a pre-grasp pose 18 cm above the object, descends to 2 cm above its center, creates a `JOINT_FIXED` constraint between the end effector and the object, and lifts to 25 cm.
- `place_and_release(drop_pos, constraint_id)`: Moves above the target, descends to 5 cm, removes the constraint, and retreats to 30 cm.

3.5 Interaction Loop (`main`)

Each iteration settles physics for 15 steps, captures the camera, lists visible objects with their positions, reads a command, resolves it to object IDs, executes pick, computes a drop pose (target position + 6 cm in z, or a default [0.4, 0, 0.05] when no place target

exists), and executes place. Typing `exit` or `quit` ends the session.

4. Implementation Details

4.1 Libraries

Library	Role
PyBullet / pybullet_data	Physics, URDF assets, rendering, IK
sentence-transformers	`all-MiniLM-L6-v2` embeddings (384-dim) and `util.cos_sim`
NumPy	Image and mask array handling
Pillow, OpenCV	Imported/listed for image utilities and future perception work
`re`	Verb-phrase extraction

4.2 Core Algorithms

Semantic matching. For a fragment embedding f and candidate embeddings $c_1 \ldots c_n$, the selected object is the owner of $\text{argmax}_i \cos(f, c_i)$. Because candidates share a color token, the decisive signal is the color word, but synonyms and phrasing variations are absorbed by the embedding space.

Inverse kinematics. PyBullet's damped-least-squares IK solves for joint angles reaching a Cartesian pose. The solution is recomputed every step, which makes motion tolerant to drift at the cost of redundant computation.

Constraint-based grasping. Instead of simulating finger contact, a fixed constraint rigidly attaches the object to the end-effector link. This removes grasp-planning difficulty and isolates the language-to-motion problem.

4.3 Design Patterns and Choices

- **Separation of concerns:** perception, grounding, and control are separate, loosely coupled units; replacing the matcher with a VLM would not touch the controller.
- **Batched encoding:** all texts are embedded in one `encode` call, minimizing model overhead per command.

- **Observability:** debug scores and printed decisions let a user audit each grounding step.
- **Graceful degradation:** regex failure falls back to whole-sentence semantics; missing placement falls back to a default drop zone.

4.4 Implementation Notes

- The pick regex alternation lists `pick` before `pick up`, so for "pick up the red cube" the extracted fragment is "up the red cube". This is harmless for embedding similarity but worth normalizing.
- The constraint is created with zero relative offsets and without explicitly aligning frames; the object may snap relative to the end-effector at the moment of attachment.
- The code indexes IK output by joint index. This is correct for the iiwa model, whose movable joints are the first seven, but is not generally valid for other URDFs.
- Fixed-step simulation uses `time.sleep(1/240)` for real-time pacing, which makes runs reproducible but slow.

5. Simulation Environment & Workflow

5.1 Environment

- **Robot:** 7-DoF KUKA LBR iiwa, fixed base, position-controlled.
- **Objects:** three cubes (red, green, blue), 6 cm edge, 0.1 kg, initially aligned along the y-axis.
- **Physics:** gravity −9.81 m/s², 240 Hz default PyBullet timestep.
- **Sensing:** one RGB + segmentation camera at 320×240.
- **Interface:** PyBullet GUI plus console text input.

5.2 Perception-to-Action Loop

1. Settle physics and render the camera frame.
2. Derive visible object IDs from the segmentation mask.
3. Receive a natural-language command.
4. Extract pick/place phrases and compute embeddings.
5. Select pick and place UIDs by cosine similarity.
6. Plan waypoints: pre-grasp → grasp → lift → above target → lower → release → retreat.

7. Execute each waypoint through IK and position control.
8. Return to step 1 for the next command.

Unlike a closed-loop VLA that re-predicts actions every few Hz from fresh images, this loop is **open-loop within a command**: perception is sampled once before execution, and object positions are queried from the simulator state rather than re-estimated.

5.3 Example Commands

- "pick up the red cube and place it on the blue cube"
- "put the blue block onto the green block"
- "grab the green thing" (no place target; uses default drop location)

6. Results, Evaluation & Discussion

No quantitative experiments were run for this report. The following describes expected behavior from code analysis and proposes an evaluation protocol.

6.1 Expected Behavior

- **Grounding:** Commands containing an explicit color word should resolve correctly, since the color token dominates similarity with the candidate descriptions. Paraphrases ("block", "colored cube", "that red one") should also resolve well.
- **Placement:** Because the grasp is a rigid constraint and the drop pose is 6 cm above the target's center, stacking is expected to succeed in most cases, subject to IK convergence and cube settling.
- **Ambiguity:** Commands that contain no color word, or that use relations the matcher cannot parse ("the cube to the left of the green one"), will fall back to whole-sentence similarity and are likely to be unreliable.

6.2 Proposed Metrics

Metric	Definition
Grounding accuracy	Fraction of commands where `pick_uid` and `place_uid` match ground truth

Metric	Definition
Task success rate	Fraction of episodes where the picked cube ends within a tolerance of the target (e.g., xy error < 2 cm, resting on top)
Paraphrase robustness	Grounding accuracy across a held-out set of rewordings per task
Margin	Gap between top-1 and top-2 similarity scores (confidence proxy)
Execution time	Wall-clock and simulation time per episode
IK residual	Distance between commanded and achieved end-effector pose

6.3 Success Criteria

An episode succeeds if (1) the correct cube is selected, (2) the cube is lifted at least 10 cm, (3) it is released over the intended target, and (4) it comes to rest on or within tolerance of the target without knocking other cubes.

6.4 Discussion

The design makes the grounding stage easy to evaluate in isolation: it can be tested offline on text alone, with no physics. Its main weakness is that it grounds *names*, not *pixels*. In a true VLA, a command like "pick the object that is the color of the sky" is resolved from appearance and world knowledge. Here it could only succeed if the embedding model happens to associate that phrase with the word "blue", and only because the object metadata already contains that word.

7. Limitations & Future Work

7.1 Limitations

- **Not a VLA:** no action tokenization, end-to-end policy, or web-scale visual pretraining.
- **Symbolic perception:** object names come from simulator metadata; the image and `seg_to_object_crops` are not used for recognition.
- **Open-loop execution:** no feedback correction, no re-grasp, no failure detection.

- **Idealized grasp:** a rigid constraint replaces gripper contact, friction, and slip.
- **Fragile parsing:** regex covers a limited verb set and sentence shapes; multi-step instructions are unsupported.
- **Single-category scene:** only three cubes, only color as a distinguishing attribute, no clutter or occlusion handling.

7.2 Sim-to-Real Challenges

- **Visual domain gap:** rendered flat-color cubes differ greatly from real textures, lighting, and sensor noise.
- **Contact dynamics:** real grippers require force control, compliance, and grasp planning; constraint grasps hide these problems.
- **Calibration:** camera extrinsics, hand-eye calibration, and kinematic errors are absent in simulation.
- **Control fidelity:** real arms need joint limits, collision avoidance, and safety monitoring beyond simple position targets.

7.3 Latency Considerations

Command grounding is cheap on CPU (a MiniLM forward pass on a handful of short strings), so the bottleneck is physical execution, not inference. A real VLA is far heavier: RT-2-scale models run at roughly 1–5 Hz and require cloud or multi-accelerator serving. Moving to such a model would require asynchronous inference, action chunking, or a low-level controller running at a higher rate than the policy.

7.4 Future Work

1. **Visual grounding:** use CLIP/SigLIP on the existing object crops so that identity comes from appearance rather than metadata.
2. **LLM/VLM parsing:** replace regex with structured output (object, relation, action) from a language model, enabling compound and relational commands.
3. **Action tokenization:** discretize end-effector deltas into tokens and train or fine-tune a small VLA on scripted demonstrations from this environment.
4. **Closed-loop control:** re-estimate object poses from depth/segmentation during execution and add retry logic.
5. **Realistic grasping:** add a parallel-jaw gripper with contact physics.

6. **Benchmarking:** randomize colors, positions, and distractors, and report the metrics in Section 6.2.
7. **Gymnasium wrapper:** expose the scene as an RL/IL environment for data collection and policy training.

References

- Brohan, A. et al. (2023). *RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control.* arXiv:2307.15818.
- Brohan, A. et al. (2022). *RT-1: Robotics Transformer for Real-World Control at Scale.* arXiv:2212.06817.
- Reimers, N. & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP.
- Coumans, E. & Bai, Y. *PyBullet, a Python module for physics simulation for games, robotics and machine learning.*
- Radford, A. et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision (CLIP).* ICML.